SOFE4790U Distributed Systems

Project Proposal: Distributed Air Traffic Control System (DATCS)

Project Proposal Group 11

Team Members

Harsh Tamakuwala	100824220
Sibi Sabesan	100750081
Hasan Khan	100820450
Armaghan Nasir	100820948
Qamar Irfan	100785387

Introduction

Air traffic control systems provide safe and efficient coordination of aircraft and flights. In the growing global airline industry, they provide consistent and accurate control of aircraft movements in increasingly congested airports by directly communicating with the aircrafts on the ground during crucial takeoff and landing phases to prevent collisions and delays. As the airline industry experiences rising air traffic volume across the world, the number and size of the aircrafts increase exponentially, so the control systems need to manage more planes, scale appropriately and maintain an acceptable level of reliability.

The objective of this project was to design a distributed air traffic control system and demonstrate concepts from distributed systems to present a somewhat decentralized model. By simulating different air control zones and their real-time communications, the system should reflect various features of a distributed architecture, such as fault tolerance, scalability and responsiveness in critical large-scale real-time systems like air traffic control at airports.

Problem Description

The traditional air traffic control system is highly centralized in nature. In a conventional architecture, the control towers are the central authorities for the airspace under their control. This provides efficient control in many cases, but comes at the cost of centralization-related drawbacks like single points of failure, low scalability, and high response times during congestion and emergency.

Proposed Approach / Solution

We will build a simulation of a distributed ATC system using modular components that reflect real-world control responsibilities. The system will feature:

- Functional Separation: Modules for airspace monitoring, ground traffic control, and instruction dispatch.
- Multicast Communication: Reliable and total order multicast (e.g., IS-IS algorithm) to synchronize control nodes.
- Concurrency & Fault Tolerance: Nodes operate concurrently and recover from failures using replication and heartbeat checks.
- Scalability: New zones can be added without disrupting existing operations.
- Simulation Interface: A GUI to visualize aircraft movement and control decisions.

This project encompasses the principles of distributed systems such as concurrency, fault tolerance, synchronization, and coordination. The lectures on failure detection, logical timestamps, and global snapshot (Chandy-Lamport) will help in designing the ordering of messages in the system, how the system should deal with failures, and ensuring system consistency. The performance of the system would be judged on the basis of correctness of coordination, recovery of failed nodes, and scaling over multiple distributed zones.

Technologies
The core development language for this project will either be Java or Python but the final choice will depend on future analysis of which language handles concurrency and simulation features more effectively. gRPC/eroMQ for node communication for messaging, Tkinter/JavaFX for the GUI simulation, and GitHub for source control. Based on our present understanding we consider this technology stack strong but acknowledge that we need to reassess as we uncover more about the project requirements and the tradeoffs during the design and implementation stages.

Realistic Timeline
We'll begin with system design and communication setup, followed by core module development and GUI simulation. Final weeks will focus on testing, polishing, and preparing the report and demo, all aligned with course deadlines.

Week	Milestone
Oct 20	Submit project proposal
Oct 21 - Oct 27	Finalize architecture and communication protocols
Oct 28 - Nov 3	Implement aircraft monitoring and ground control modules
Nov 4 - Nov 10	Integrate multicast protocols and fault tolerance
Nov 11 - Nov 17	Develop GUI and simulation interface
Nov 18 - Nov 22	Testing, performance evaluation, final polish
Nov 23	Submit code and final report
Nov 24 - Dec 1	Project demo and presentation

Team Members and Responsibilities
All members of the team will be involved in all aspects of the project from design through implementation, testing and documentation. However, for the purposes of clarity and accountability one member of the team will be the primary responsible for each area:

Member	**Responsibilities**
Sibi Sabesan	Documentation, evaluation metrics, final report
Hasan Khan	GUI simulation, user interface
Armaghan Nasir	Aircraft tracking logic, concurrency mechanisms
Harsh Tamakuwala	Multicast implementation, coordination logic, testing
Qamar Irfan	Fault tolerance mechanisms, heartbeat protocol